Project: SPECK32/64 Lightweight Block Cipher — Datapath + Controller Design

Duration: 2 days

1. Background

SPECK is an NSA-published lightweight block cipher family designed for constrained hardware. Unlike AES (S-box tables, GF(2^8) MixColumns), SPECK is an **ARX cipher** — every round uses only **A**ddition (mod 2^n), **R**otation, and **X**OR. No lookup tables, no finite-field math — just an adder, two rotators, and XOR gates around a register. It's the same round-based-cipher architecture (datapath + round-counting FSM) as AES or PRESENT, but small enough to build in 2 days.

You will implement **SPECK32/64**: 32-bit block, 64-bit key, 22 rounds.

2. Parameters

Parameter	Value
Word size (n)	16 bits
Block = two words	(x, y), 16 bits each
Key = four words	(k_3, k_2, k_1, k_0), 16 bits each
Rounds (T)	22
Right rotation (α)	7
Left rotation (β)	2

3. Round Function

For round i = 0 … 21, given round key k_i:

```
x_(i+1) = ( ROTR(x_i, 7) + y_i ) XOR k_i
y_(i+1) = ROTL(y_i, 2) XOR x_(i+1)
```

- + is addition mod 2^{16} (unsigned add, discard carry-out)
- ROTR/ROTL are fixed-amount bit rotations — implement as wiring, not a barrel shifter or loop
- Ciphertext = (x, y) after round 21

4. Key Schedule

Round keys are generated using the **same round function**, with the round index i playing the role of the round key. Initialize RK[0] = k_0, l[0]=k_1, l[1]=k_2, l[2]=k_3. For i = 0 … 20:

```
l[i+3]  = ( ROTR(l[i], 7) + RK[i] ) XOR i
RK[i+1] = ROTL(RK[i], 2) XOR l[i+3]
```

Note this is structurally identical hardware to the encryption round — a deliberate design decision point: reuse one ALU (mux-shared between data round and key expansion) or instantiate two. Justify your choice.

5. What to Implement

You must build a **datapath** (the arithmetic/storage hardware: adders, rotators, XORs, and whatever registers you decide you need) and a **controller** (an FSM that sequences the datapath through 22 rounds and manages the start/done handshake). Beyond that split, the internal architecture is your design decision:

- How many registers, and what each one holds, is up to you — as long as the datapath correctly reproduces the round function and key schedule across 22 rounds.
- How many FSM states you need, and what you name them, is up to you — as long as the controller correctly sequences reset → load → 22 rounds → output-valid → ready for next operation.
- Whether you share one ALU between the encryption round and the key schedule, or build two separate datapaths for them, is your call — justify the tradeoff in your report.
- Whether the key schedule computes all round keys up front (before encryption starts) or generates them on the fly, one per cycle alongside encryption, is also your call. Justify the trade-off.

The only fixed requirements are the **algorithm** (must match exactly).

```
module speck32_64_top (
  input  logic         clk, rst_n,
  input  logic         start,
  input  logic [63:0]  key_in,     // {k3,k2,k1,k0}
  input  logic [31:0]  plaintext,  // {x0,y0}
  output logic [31:0]  ciphertext,
  output logic         valid_out
);
```

Use a proper start/valid_out handshake. Ports of top level module are given above for reference.

6. Testing Plan

1. **Reference model:** write a short Python/C model of the algorithm above — this is your golden model for generating test vectors.
2. **Official test vector (mandatory):**

- key = 64'h1918_1110_0908_0100
- plaintext = 32'h6574_694c
- expected ciphertext = 32'ha868_42f2 Testbench must self-check this and print

PASS/FAIL.

1. **Random vectors:** generate 50–100 random key/plaintext pairs from your reference model, compare agains t RTL output.

Done means: self-checking testbench output (PASS + vector count summary).

Note: Design **diagram first**, **code second.** Draw the complete hardware architecture and interface signals before writing any RTL.